

30-dniowy plan INF.04 — Tydzień 1: Fundament OOP + WPF

Zadania próbuj rozwiązywać samodzielnie bez pomocy asystentów kodowania (jeżeli już musisz, to rób to z głową). Nazwy klas, metod i zmiennych dobierasz sam — oceniana jest ich sensowność. Nagłówki dokumentacyjne są obowiązkowe nad każdą metodą.

● Java ● C# / WPF ● Logika + pliki

7 DNI	14 ZADAŃ	4+3 JAVA + WPF	~10h ŁĄCZNY CZAS
-----------------------------	--------------------------------	--------------------------------------	--

Zadanie 1.1 — System zarządzania flotą pojazdów

Zbuduj klasę reprezentującą pojazd w firmie leasingowej. Klasa musi chronić swoje dane — żaden kod zewnętrzny nie może ustawić przebiegu na wartość ujemną ani roku sprzed 1900.

WYMAGANIA

- Cztery pola prywatne: numer rejestracyjny (String), rodzaj pojazdu (String: dozwolone wartości to osobowy, dostawczy, motocykl), rok produkcji (int), przebieg (double).
- Konstruktor sprawdza wszystkie dane przy tworzeniu obiektu — jeśli coś jest nie tak, wypisuje odpowiedni komunikat.
- Rok produkcji musi mieścić się w przedziale 1900–rok bieżący. Do pobrania aktualnego roku użyj klasy Calendar lub LocalDate.
- Metoda sprawdzająca, czy pojazd wymaga serwisu — warunek: wiek pojazdu powyżej 5 lat LUB przebieg powyżej 20 000 km.
- Metoda zwracająca tekstowy opis pojazdu (nadpisz toString).
- Gettery dla wszystkich pól; setter tylko dla przebiegu, z walidacją.

WZORZEC WYJŚCIA (TWOJE WARTOŚCI MOGĄ SIĘ RÓŻNIĆ)

```
[WZ12345] osobowy, 2018r., 15 230,0 km — SERWIS: NIE
[KR99887] dostawczy, 2014r., 48 120,5 km — SERWIS: TAK
[WA00001] motocykl, 1999r., 2 300,0 km — SERWIS: TAK
```

Dodatkowe wyzwanie: Dopisz metodę statyczną, która tworzy obiekt Pojazd z jednej linii tekstowej w formacie TXT: WZ12345;osobowy;2018;15230.0. Użyj metody split(";") i odpowiednich metod konwersji.

Zadanie 1.2 — Raport serwisowy floty

Program główny zarządza kolekcją pojazdów i generuje raport dla działu serwisu.

WYMAGANIA

- Utwórz kolekcję (ArrayList) z minimum 6 pojazdami — różne typy, różne roczniki i przebiegi.
- Napisz metodę statyczną przyjmującą kolekcję i aktualny rok, która wypisuje wyłącznie pojazdy wymagające serwisu.
- Wyświetl nagłówek raportu z datą oraz podsumowanie: ile pojazdów wymaga serwisu i jaki to procent całej floty.
- Posortuj listę po roku produkcji rosnąco (użyj Collections.sort z Comparatorem).

WZORZEC WYJŚCIA (TWOJE WARTOŚCI MOGĄ SIĘ RÓŻNIĆ)

```
=== RAPORT SERWISOWY — 2025 ===
Pojazdy wymagające serwisu (4 z 6, 67%):
[KR99887] dostawczy, 2014r., 48120.5 km
[GD44321] osobowy, 2019r., 21050.0 km
...
=====
```

KRYTERIA OCENIANIA

4 pkt

Pola prywatne, gettery/settery, hermetyzacja

3 pkt

Walidacja danych w konstruktorze (możesz dodać obsługę wyjątku)

3 pkt

Poprawna logika warunku serwisowego

2 pkt

Nagłówki dokumentacyjne nad każdą metodą

Wskazówka egzaminatora: Na egzaminie zawsze zacznij od klasy danych (Pojazd), potem klasę zarządzającą, na końcu main(). Nigdy nie pisz całej logiki w main().

Częsty błąd: Porównanie Stringów przez == działa tylko dla literałów. Przy obiektach zawsze używaj .equals() — np. przy sprawdzaniu rodzaju pojazdu.

Zadanie 2.1 — Rejestr egzaminatorów z unikalnym ID

Zaprojektuj klasę Egzaminator tak, aby każdy nowy obiekt automatycznie otrzymywał kolejny numer ID — bez żadnej ingerencji z zewnątrz. Mechanizm ma działać niezależnie od kolejności tworzenia obiektów.

WYMAGANIA

- Jedno prywatne, statyczne pole całkowitoliczbowe pełni rolę licznika — jest inkrementowane w konstruktorze przy każdym nowym obiekcie.
- Pole id jest final — ustawiane raz w konstruktorze, nie może być zmienione z zewnątrz.
- Pola: imię, nazwisko, specjalizacja. Specjalizację zaimplementuj jako enum z wartościami INF02, INF03, INF04. Enum powinien mieć metodę zwracającą sformatowaną nazwę (np. "INF.03").
- Metoda wyświetlająca dane formatuje je tabelarycznie — numer ID zawsze na 3 cyfry (z zerami wiodącymi: 001, 002...).

WZORZEC WYJŚCIA (TWOJE WARTOŚCI MOGĄ SIĘ RÓŻNIĆ)

```
[ID:001] Anna Nowak      | INF.02
[ID:002] Piotr Lisowski  | INF.04
[ID:003] Ewa Wąsowska   | INF.03
```

Zadanie 2.2 — Dowód działania mechanizmu ID

Napisz program testujący, który w czytelny sposób dokumentuje działanie licznika — tak jakbyś tłumaczył to egzaminatorowi. Program musi jednoznacznie pokazać, że ID jest nadawane automatycznie i rośnie o 1.

WYMAGANIA

- Wypisz stan licznika PRZED utworzeniem pierwszego obiektu.
- Twórz obiekty jeden po drugim — po każdym wypisz komunikat z aktualnym ID i stanem licznika.
- Wypisz stan licznika PO wszystkich obiektach.
- Wypisz potwierdzenie, że obiekt 2 ma ID o 1 większe od obiektu 1 itd.

WZORZEC WYJŚCIA (TWOJE WARTOŚCI MOGĄ SIĘ RÓŻNIĆ)

```
Licznik przed tworzeniem: 0
--- Tworzę egzaminatora #1 ---
Dane: [ID:001] Anna Nowak | INF.02
Licznik teraz: 1
--- Tworzę egzaminatora #2 ---
Dane: [ID:002] Piotr Lisowski | INF.04
Licznik teraz: 2
--- Tworzę egzaminatora #3 ---
Dane: [ID:003] Ewa Wąsowska | INF.03
Licznik teraz: 3
Licznik po zakończeniu: 3
Różnica ID: obj2 - obj1 = 1 ✓
```

Dodatkowe wyzwanie: Użyj tablicy dwuwymiarowej z danymi testowymi zamiast powielać kod tworzenia obiektów.

3 pkt

Poprawne użycie static dla licznika instancji

3 pkt

Pole id jest final, niemodyfikowalne z zewnątrz

2 pkt

Enum Specjalizacja z metodą formatującą nazwę

4 pkt

Czytelne wyjście udowadniające mechanizm ID

Wskazówka egzaminatora: Pole static należy do klasy, nie do obiektu. Nawet jeśli stworzysz tysiąc obiektów i program je usunie, licznik nie wraca sam do 0. Ta różnica między static a zwykłym polem jest kluczowa.

Częsty błąd: Nie deklaruj licznikInstancji jako public — każdy mógłby wtedy dowolnie go zmienić. Udostępnij go wyłącznie przez getter.

Zadanie 3.1 — Import produktów z pliku z weryfikacją kodu

Wczytaj dane z dostarczonego pliku produkty.txt i stwórz listę obiektów. Każdy produkt zawiera kod partii złożony z 11 cyfr — ostatnia jest cyfrą kontrolną obliczaną algorytmem. Program musi odróżnić kody poprawne od sfalszowanych.

WYMAGANIA

- Format pliku: każda linia to Nazwa;Cena;Ilosc;KodPartii. Plik zawiera 11 wierszy, w tym jeden z błędną ceną (BRAK) — taki wiersz pomijasz z komunikatem ostrzeżenia, nie przerywasz programu.
- Odczyt pliku: użyj klasy `BufferedReader` opakowanej w blok `try-with-resources`.
- Walidacja kodu partii — kod składa się z 11 cyfr. Weź pierwsze 10 cyfr i przemnoż każdą przez odpowiednią wagę z ciągu: 1, 3, 7, 9, 1, 3, 7, 9, 1, 3. Zsumuj wyniki. Oblicz resztę z dzielenia sumy przez 10. Jeśli reszta wynosi 0, cyfra kontrolna (11. cyfra) powinna być 0. W przeciwnym razie powinna wynosić 10 minus reszta. Porównaj z rzeczywistą 11. cyfrą.
- Klasa produktu zawiera metodę sprawdzającą poprawność kodu — zwraca wartość logiczną.
- Wyodrębnij metodę obliczającą samą sumę kontrolną — nie rób wszystkiego w jednej metodzie.

WZORZEC WYJŚCIA (TWOJE WARTOŚCI MOGĄ SIĘ RÓŻNIĆ)

[POMINIĘTO] Mąka Pszena — błędna cena: "BRAK"

Wczytano 10 produktów.

Produkty z POPRAWNYM kodem (6):

Jabłka	2,49 zł	150 szt	kod: 10433218198
Mleko 3.2%	3,19 zł	80 szt	kod: 60013389082
Masło Ekstra	7,99 zł	30 szt	kod: 54235116157
Jogurt Naturalny	2,79 zł	120 szt	kod: 49593103413
Herbata Liściasta	12,99 zł	25 szt	kod: 31647525531
Ser Gouda	18,50 zł	40 szt	kod: 83503056419

Produkty z BŁĘDNYM kodem (4):

Chleb Pszeny, Ryż Basmati, Kawa Ziarnista, Cukier Biały

Zadanie 3.2 — Generowanie etykiety magazynowej

Do klasy produktu dopisz metodę tworzącą etykietę skróconą — używaną w systemach magazynowych do oszczędzania miejsca na wydruku. Etykieta powstaje przez dwuetapowe przekształcenie nazwy.

WYMAGANIA

- Krok 1 — usuń sąsiadujące duplikaty znaków: jeśli dwa takie same znaki stoją obok siebie, zostaw tylko jeden. Przykład: "Appple" → "Apple", "Chlebb" → "Chleb".
- Krok 2 — zamień wszystkie samogłoski (a, e, i, o, u oraz ich polskie odpowiedniki: ą, ę, ó) na gwiazdkę. Zamiana dotyczy zarówno małych, jak i wielkich liter.
- Kolejność kroków jest nieodwracalna — najpierw duplikaty, potem samogłoski. Zamienione w złej kolejności dają inny wynik.
- Wyświetl etykiety wyłącznie dla produktów z poprawnym kodem partii.

WZORZEC WYJŚCIA (TWOJE WARTOŚCI MOGĄ SIĘ RÓŻNIĆ)

Etykiety produktów (poprawny kod):

Jabłka	→ J*błk*
Mleko 3.2%	→ Młk* 3.2%
Masło Ekstra	→ M*sł* *kstr*
Jogurt Naturalny	→ J*g*rt N*t*r*lny

Herbata Liściasta → H*rb*t* L*śc**st*
Ser Gouda → S*r G**d*

Dodatkowe wyzwanie: Policz wartość magazynową wyłącznie poprawnych produktów (cena × ilość) i wyświetl ją na końcu raportu.

KRYTERIA OCENIANIA

5 pkt

Poprawna implementacja algorytmu sumy kontrolnej

3 pkt

Poprawna etykieta — właściwa kolejność kroków

3 pkt

Odczyt pliku z try-with-resources i obsługą błędów

2 pkt

Wydzielona metoda do obliczania sumy (nie wszystko w jednej)

Wskazówka egzaminatora: Kolejność w etykiecie jest krytyczna. Sprawdź ją na papierze dla słowa "Appple": duplikaty → "Apple", samogłoski → "*ppl*". Odwrotnie: samogłoski → "*ppl*", duplikaty → "*pl*" — to inny wynik!

Częsty błąd: Przy odczycie pliku zawsze używaj try-with-resources: try (BufferedReader br = ...) { }. To gwarantuje zamknięcie pliku nawet gdy wystąpi wyjątek w środku.

Zadanie 4.1 — Trzy typy kont z własną logiką wypłat

Zbuduj hierarchię kont bankowych. Każdy typ konta reaguje inaczej na tę samą operację wypłaty i wpłaty. Logika musi być w klasach — nie w main().

WYMAGANIA

- Klasa bazowa posiada dwa pola: numer konta i saldo. Metoda wypłaty blokuje operację, jeśli saldo po wypłacie byłoby ujemne — wypisuje komunikat i zwraca false. Metoda wpłaty waliduje, że kwota jest większa od zera.
- KontoOszczednosciowe — nadpisuje metodę wypłaty: pobiera dodatkowo prowizję 5 zł. Jeśli łączna kwota (wypłata + prowizja) przekracza saldo, operacja jest blokowana. Nadpisuje metodę wpłaty: dolicza 1% bonusu do wpłacanej kwoty.
- KontoPremium — posiada dodatkowe pole: limit debetu. Pozwala na ujemne saldo, ale tylko do wartości limitu. Posiada metodę naliczającą odsetki karne: jeśli saldo jest ujemne, pobiera 10% od kwoty debetu.
- Każda klasa nadpisuje toString() zwracając typ konta, numer i aktualne saldo.

WZORZEC WYJŚCIA (TWOJE WARTOŚCI MOGĄ SIĘ RÓŻNIĆ)

```
[Konto]                PL001 | saldo: 1000,00 zł
[KontoOszczednosciowe] PL002 | saldo: 1000,00 zł
[KontoPremium]         PL003 | saldo: 1000,00 zł limit: 500,00 zł
```

Zadanie 4.2 — Symulacja operacji na tablicy kont

Tablica typu bazowego przechowuje obiekty różnych typów. Ta sama pętla wykonuje te same operacje na każdym koncie — efekty są różne, bo każde konto ma swoją logikę.

WYMAGANIA

- Utwórz tablicę typu bazowego z trzema obiektami: zwykłe konto, oszczędnościowe, premium. Każde zaczyna z saldem 1000 zł.
- W jednej pętli: wpłać 1000 zł, potem wypłać 1200 zł, następnie jeśli konto jest typu KontoPremium — wywołaj naliczanie odsetek (użyj instanceof do sprawdzenia).
- Po operacjach wypisz typ konta, numer i końcowe saldo.
- Jeśli wypłata nie doszła do skutku, wypisz czytelny komunikat dlaczego.

WZORZEC WYJŚCIA (TWOJE WARTOŚCI MOGĄ SIĘ RÓŻNIĆ)

```
--- Konto PL001 ---
Wpłata 1000 zł → saldo: 2000,00 zł
Wypłata 1200 zł → saldo: 800,00 zł
Końcowe saldo: 800,00 zł

--- KontoOszczednosciowe PL002 ---
Wpłata 1000 zł (+1%) → saldo: 2010,00 zł
Wypłata 1200 zł (+5 zł prowizji) → saldo: 805,00 zł
Końcowe saldo: 805,00 zł

--- KontoPremium PL003 ---
Wpłata 1000 zł → saldo: 2000,00 zł
Wypłata 1200 zł → saldo: 800,00 zł
Odsetki karne: 0,00 zł (saldo dodatnie)
Końcowe saldo: 800,00 zł
```


Dodatkowe wyzwanie: Dodaj metodę statyczną obliczającą sumę sald wyłącznie dla kont z saldem dodatnim. Wywołaj ją po pętli i wypisz wynik.

KRYTERIA OCENIANIA

4 pkt

Poprawna hierarchia z `@Override` w każdej klasie pochodnej

4 pkt

Poprawna logika każdego z trzech typów kont

3 pkt

Polimorficzna pętla — brak `if/else` sprawdzającego typ

2 pkt

`instanceof` użyty tylko tam, gdzie naprawdę potrzebny

Wskazówka egzaminatora: Polimorfizm oznacza: piszesz `konto.wypłac(1200)` i nie musisz wiedzieć, jakie to konto — JVM sam wybierze właściwą metodę. Jeśli widzisz dużo `instanceof` i rzutowań, to sygnał, że coś jest nie tak z projektem.

Częsty błąd: W klasie pochodnej możesz wywołać metodę z klasy bazowej przez `super.wplac(kwota)`. Nie musisz przepisywać całej logiki — tylko ją rozszerz.

Zadanie 5.1 — Interaktywny rejestr egzaminatorów

Przenieś logikę klasy Egzaminator z Dnia 2 do aplikacji WPF. Interfejs umożliwia wyszukiwanie po ID oraz dodawanie nowych egzaminatorów. Przycisk dodawania jest aktywny tylko gdy pola są wypełnione.

WYMAGANIA

- Tło okna: CadetBlue (#5F9EA0). Wymiary: 460 × 440 px. Tytuł okna: Rejestr Egzaminatorów.
- Sekcja wyszukiwania: pole tekstowe na ID, przycisk wyszukiwania, etykieta na wynik pod przyciskiem.
- Jeśli egzaminator został znaleziony: etykieta pokazuje imię, nazwisko i specjalizację. Kolor tekstu: ciemnozielony.
- Jeśli egzaminator nie istnieje: etykieta pokazuje komunikat Błąd: Egzaminator o ID [X] nie widnieje w systemie. Kolor tekstu: czerwony.
- Jeśli wpisano coś innego niż liczbę: osobny komunikat o błędnym formacie — kolor: pomarańczowy.
- Sekcja dodawania: pola na imię i nazwisko, lista rozwijana ze specjalizacjami (INF.02, INF.03, INF.04), przycisk dodawania.
- Przycisk dodawania jest zablokowany (IsEnabled = false) gdy przynajmniej jedno z pól — imię lub nazwisko — jest puste. Aktualizacja przy każdej zmianie tekstu.
- Lista wszystkich egzaminatorów widoczna w ListBox. Po dodaniu nowego lista odświeża się automatycznie.
- Przy starcie aplikacja ma 5 predefiniowanych egzaminatorów z różnymi ID i specjalizacjami.

Dodatkowe wyzwanie: Po wybraniu egzaminatora na liście (zdarzenie SelectionChanged) pola imienia i nazwiska w sekcji dodawania automatycznie się wypełniają — pomocne przy edycji.

KRYTERIA OCENIANIA

3 pkt

Poprawny układ XAML z nazwanymi kontrolkami

3 pkt

Wyszukiwanie — trzy warianty kolorystyczne komunikatu

2 pkt

Blokada przycisku działająca dynamicznie przy TextChanged

2 pkt

Dodawanie i automatyczne odświeżanie ListBox

2 pkt

Logika klasy Egzaminator (static ID) przeniesiona z Dnia 2

Wskazówka egzaminatora: Blokowanie przycisku realizuj w zdarzeniu TextChanged obu pól tekstowych. Warunek: `IsEnabled = !string.IsNullOrEmpty(txtImie.Text) && !string.IsNullOrEmpty(txtNazwisko.Text)`.

Częsty błąd: Klasa Egzaminator z Dnia 2 była w Javie — teraz przepisujesz ją w C#. Gettery/settery zastąp właściwościami (properties). Mechanizm static int licznik działa identycznie.

Zadanie 6.1 — Podgląd magazynu z walidacją kodów

Aplikacja wczytuje dane z pliku produkty.txt automatycznie przy starcie. Użytkownik widzi listę produktów i może kliknąć dowolny, aby zobaczyć szczegóły — status kodu partii i etykietę magazynową.

WYMAGANIA

- Dane są wczytywane przy uruchomieniu programu (np. w konstruktorze po InitializeComponent lub w zdarzeniu Loaded).
- Jeśli plik nie istnieje lub nie można go odczytać, wyświetl stosowny komunikat w interfejsie — aplikacja nie może się wysypać.
- Okno podzielone pionowo: lewa strona to ListView z kolumnami Nazwa, Cena, Ilość, Status kodu. Prawa strona to panel szczegółów — początkowo pusty.
- Kolumna Status kodu pokazuje tekst Poprawny lub Błędny. Użyj konwertera lub ustaw kolor w zdarzeniu.
- Kliknięcie produktu na liście (SelectionChanged) wypełnia panel prawy: pełna nazwa produktu, status kodu (zielony/czerwony), etykieta magazynowa czcionką monospace.
- Na dole okna pasek statusu wyświetla: Załadowano X produktów — Y z poprawnym kodem.

Dodatkowe wyzwanie: Dodaj możliwość filtrowania listy po nazwie: pole tekstowe nad ListView, wpisanie frazy na bieżąco zawęży listę (zdarzenie TextChanged, bez przycisku).

KRYTERIA OCENIANIA

3 pkt

Ładowanie danych przy starcie
— bez przycisku

3 pkt

ListView z kolumnami i
wiązaniami danych (XAML)

2 pkt

SelectionChanged z
wypełnieniem panelu
szczeółów

2 pkt

Kolorowanie statusu + pasek
statusu na dole

2 pkt

Obsługa braku pliku bez crash aplikacji

Wskazówka egzaminatora: Dane do ListView podaj przez ItemsSource. Stwórz ObservableCollection<Produkt> jako pole klasy — automatycznie odświeży widok gdy dodasz lub usuniesz element.

Częsty błąd: ListView z GridViewColumn wymaga definicji kolumn w XAML przez DisplayMemberBinding. Nie próbuj dodawać kolumn z kodu — to trudne i zbędne. Zrób to w XAML.

Zadanie 7.1 — System koszyka zakupowego z logiką rabatową

Zbuduj system zamówień oparty na agregacji: obiekt zamówienia zawiera w sobie kolekcję pozycji. Logika obliczania sumy, rabatów i czyszczenia koszyka należy do klasy zamówienia — nie do main().

WYMAGANIA

- Klasa pozycji ma trzy pola: nazwa, cena jednostkowa, ilość.
- Klasa zamówienia zawiera prywatną listę pozycji oraz numer zamówienia generowany automatycznie (np. przez statyczny licznik lub UUID).
- Pozycje dodaje się wyłącznie przez dedykowaną metodę — lista nie jest dostępna bezpośrednio z zewnątrz (hermetyzacja).
- Metoda obliczająca sumę: iteruje po pozycjach i mnoży cenę przez ilość każdej z nich.
- Metoda rabatowa: przyjmuje frazę tekstową i obniża cenę o 20% tylko tym pozycjom, których nazwa zawiera tę frazę (bez rozróżniania wielkości liter).
- Metoda czyszcząca: usuwa z listy pozycje z ilością równą 0. Nie wolno używać pętli for-each razem z remove() — to grozi wyjątkiem ConcurrentModificationException. Użyj removeIf().
- Metoda wyświetlająca podsumowanie: lista pozycji, separator, suma zamówienia i numer.

WZORZEC WYJŚCIA (TWOJE WARTOŚCI MOGĄ SIĘ RÓŻNIĆ)

```
=== ZAMÓWIENIE #0001 ===
  Jabłka           x 3 | 7,47 zł
  Mleko 3.2%       x 2 | 6,38 zł
  Chleb Pszenny    x 1 | 4,29 zł ← rabat 20% (frazą: "Chleb")
  Kawa Ziarnista   x 0 | (usunięta)
-----
SUMA: 18,14 zł
```

Dodatkowe wyzwanie: Dopisz metodę zwracającą pozycję o najwyższej wartości (cena × ilość). Użyj stream().max() z odpowiednim komparatorem.

Zadanie 7.2 — Kalkulator kosztów dostawy (WPF)

Interfejs pozwala wybrać opcje dostawy i wyświetla pełne podsumowanie zamówienia powiększone o koszt wysyłki.

WYMAGANIA

- Dwie sekcje zgrupowane: wybór rodzaju przesyłki przez RadioButton (List — 5 zł, Paczka — 15 zł) oraz opcje dodatkowe przez CheckBox (Ubezpieczenie — +3 zł, Priorytet — +10 zł).
- Przycisk obliczający koszt: zbiera zaznaczone opcje, sumuje koszty dostawy, dodaje do sumy zamówienia z Zadania 7.1.
- Pole tekstowe tylko do odczytu (lub TextBlock) pokazuje podsumowanie zamówienia — listę pozycji i częściową sumę.
- Etykieta końcowa wyświetla: Suma zamówienia: X,XX zł + Dostawa: Y,YY zł = Łącznie: Z,ZZ zł.
- Przycisk jest nieaktywny jeśli żaden RadioButton nie jest zaznaczony.

WZORZEC WYJŚCIA (TWOJE WARTOŚCI MOGĄ SIĘ RÓŻNIĆ)

```
Suma zamówienia: 18,14 zł
Dostawa: Paczka (15,00 zł) + Ubezpieczenie (3,00 zł)
= Łącznie: 36,14 zł
```

Dodatkowe wyzwanie: Pokoloruj etykietę końcową: zielony jeśli łącznie < 50 zł, pomarańczowy 50–150 zł, czerwony powyżej 150 zł.

KRYTERIA OCENIANIA

3 pkt

Poprawna agregacja — lista prywatna, dostępna przez metodę

3 pkt

Metody rabatu i czyszczenia

2 pkt

WPF — RadioButton i CheckBox poprawnie sumowane

2 pkt

Wyświetlenie kompletnego podsumowania

2 pkt

Nagłówki dokumentacyjne nad każdą metodą

Wskazówka egzaminatora: Agregacja to NIE dziedziczenie. Zamówienie NIE jest pozycją — Zamówienie MA listę pozycji. To jeden z fundamentów dobrego projektowania obiektowego.

Częsty błąd: Pętla for-each z remove() to ConcurrentModificationException w czasie wykonania. Zawsze używaj removeIf(warunek) — jest bezpieczna i czytelna.